JAIN DEEMED-TO-BE UNIVERSITY | SCHOOL OF COMPUTER SCIENCE AND IT

Department of CS and IT

4th Semester

MOBILE COMPUTING

(23BCAM4C01)

Activity 2

Smart Q&A: An Innovative Doubt-Box Application

with Upvoting and Anonymous Questioning

SUBMITTED TO

Dr. RAMKUMAR KRISHNAMOORTHY

Submitted by:

Vishak V-24BCAR0100

JGi JAIN DEEMED-TO-BE UNIVERSITY

CERTIFICATE

This is to certify that **Mr. Vishak V** has satisfactorily completed **Activity-2** on 03/05/2026 prescribed by JAIN Deemed to be university for the **Fourth Semester** Degree course in the year 2026-2027.

S. No.	CRITERIA	MARKS	MARKS OBTAINED
1	Abstract	3	
2	Implementation	10	
3	Diagram	3	
4	Report/Views	4	
5	Viva	3	
6	On-Time Submission	2	
	TOTAL	25	

MARKS	
Total	Conversion of Marks to 15
25	

Signature of the Student

Signature of the Staff member

JGi JAIN DEEMED-TO-BE UNIVERSITY

TABLE OF CONTENTS

SECTION	PAGE
1. Abstract & Introduction	4-5
2. Implementation Scenario in Real-Time	6-7
3.Architecture & System Design	8-9
4. Key Features & Modules	10-11
5.Diagram	12
6. Your Views & Concluding Remarks	13-15
7. References	16

1.ABSTRACT

The Smart Q&A application is an Android-based mobile platform designed to bridge the communication gap between students and faculty members in an academic environment. The app provides an innovative, interactive doubt-resolution system that goes beyond a traditional "doubt box" by incorporating real-time question posting, anonymous questioning, upvoting of doubts by peers, and a faculty response dashboard. Built using Android Studio with Java and backed by Firebase (Firestore and Authentication), the application enables seamless interaction across two distinct user roles: students and faculty.

The core objective of this project is to reduce hesitation among students in asking questions in classroom settings and to help faculty prioritize the most common doubts through a democratic upvoting mechanism. This report documents the concept, design, implementation, and personal reflections on the application.

1.1 INTRODUCTION

In traditional academic settings, students often feel reluctant to ask questions openly in a classroom due to peer pressure, fear of judgment, or simply due to time constraints. A physical "doubt box" partially addresses this by allowing anonymous written submissions, but it lacks the interactivity, real-time responsiveness, and analytical capability needed in the modern digital era.

Smart Q&A transforms this concept into a dynamic mobile application. The application enables:

- Students can post questions either with their identity visible or completely anonymously.
- Peers to upvote doubts they share, pushing the most common questions to the top.
- Faculty to view a prioritized, real-time feed of questions and respond with answers directly through the app.
- Both parties to track resolved and pending questions through clear status badges.

This project aligns with the Mobile Computing course (23BCAM4C01) as it demonstrates core mobile development concepts including Activity lifecycle management, Firebase Realtime Database integration, RecyclerView adapters, Intent-based navigation, and role-based authentication — all implemented in an Android Studio environment using Java.

2. IMPLEMENTATION SCENARIO IN REAL-TIME USAGES

1. Real-World Use Cases

The Smart Q&A application has direct and practical real-world applications in educational institutions:

Use Case 1 — Classroom Doubt Resolution

During or after a lecture, students often accumulate questions. With Smart Q&A, a student opens the app, taps "Ask a Question," selects the subject (e.g., Mobile Computing), types their doubt, and optionally checks the Anonymous toggle before submitting. Within seconds, the question appears in the faculty dashboard, sorted by upvotes. If five students upvote the same question, it rises to the top of the faculty's feed — giving the instructor an immediate view of what the class struggles with most.

Use Case 2 — Examination Preparation

Before exams, students tend to post high volumes of conceptual questions. Faculty can log in, filter questions by subject, see which topics received the most upvotes, and post concise answers. Students receive live answers directly within the same card that displays their question — eliminating the need for separate WhatsApp groups or email threads.

Use Case 3 — Anonymous Feedback Channel

Sometimes students wish to report syllabus concerns, unclear teaching, or request topic revision without exposing their identity. The anonymous mode allows this safely. The faculty sees the question with a lock icon and "Anonymous" label — they know the concern is genuine without knowing who raised it.

Technology Stack

Technology	Usage
Android Studio	IDE for development and emulator testing
Java	Primary programming language
Firebase Authentication	Email/password login for students and faculty
Cloud Firestore	Real-time NoSQL database for questions
RecyclerView	Efficient list rendering of questions
CardView	Material Design question cards
SharedPreferences	Storing user role (student/faculty) locally

3.ARCHITECTURE & SYSTEM DESIGN

Application Architecture

The Smart Q&A application follows the Client-Server architecture pattern where the Android app acts as the client and Firebase acts as the backend server. The architecture consists of three main layers:

- Presentation Layer: XML layouts and Java Activity classes managing the UI.
- Business Logic Layer: Java classes handling upvoting logic, role detection, and question state transitions.
- Data Layer: Cloud Firestore as the NoSQL real-time database, with Firebase Authentication for user management.

Data Flow Diagram

The following describes the data flow within the application:

1. The user opens the app → Role selection screen (Student / Faculty).
2. User authenticates via Firebase Authentication (email + password).
3. Student: navigates to StudentDashboardActivity → views questions ordered by upvotes.
4. Student taps "Ask a Question" → AskQuestionActivity → fills question form → submits to Firestore.
5. Faculty: navigates to FacultyDashboardActivity → views all questions with filter options.
6. Faculty taps "Answer" → an AlertDialog input appears → answer stored back to Firestore.
7. Both student and faculty dashboards use Firestore's addSnapshotListener for real-time updates.

Firestore Data Structure

All questions are stored in the 'questions' collection in Cloud Firestore with the following fields:

Field	Type	Description
text	String	The question content
subject	String	Subject tag (e.g., Mobile Computing)
anonymous	Boolean	True if asked anonymously
askedBy	String	Email or 'Anonymous'
userId	String	Firebase UID of the asker
upvotes	Integer	Current upvote count
upvotedBy	Array	List of UIDs who upvoted
answered	Boolean	Whether faculty has answered
answer	String	Faculty's answer text
status	String	'pending' or 'answered'
timestamp	Date	Time of question submission

4.KEY FEATURES & MODULES

Dual Role System

The application supports two distinct roles managed through Firebase Authentication and Android SharedPreferences. Students and faculty are presented separate dashboards with role-appropriate features. This separation ensures that faculty cannot accidentally upvote questions and students cannot accidentally post answers.

Anonymous Questioning

The anonymity feature is implemented at the application level: when a student checks the "Ask Anonymously" checkbox, the askedBy field is set to the string "Anonymous" and the Firebase UID is still stored (for deduplication of upvotes) but never displayed. The faculty dashboard shows a lock icon next to anonymous questions, making the interface both intuitive and privacy-conscious.

Upvoting System

The upvote mechanism uses Firestore's FieldValue.arrayUnion() and arrayRemove() methods to maintain a list of user IDs (upvotedBy) who have voted on each question. This ensures each user can upvote a question only once and can also undo their upvote. Questions are sorted in descending order by upvotes, ensuring the faculty sees the most pressing doubts first.

Real-Time Synchronization

Both dashboards use Firestore's addSnapshotListener, which provides live WebSocket-based updates. When a faculty member posts an answer, students see the answer appear on their screen within milliseconds without needing to refresh the page. Similarly, when a new question is posted, the faculty dashboard updates instantly.

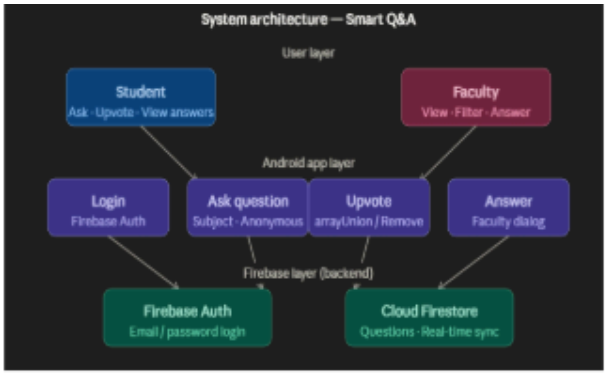
Subject Filtering (Faculty)

The faculty dashboard includes three filter buttons — All, Pending, and Answered — allowing instructors to quickly focus on unresolved questions. This improves workflow efficiency for faculty who may be managing multiple subjects simultaneously.

Activity Modules Summary

Activity / File	Responsibility
MainActivity.java	Role selection & auto-login redirect
LoginActivity.java	Firebase email/password auth for both roles
StudentDashboardActivity.java	View questions sorted by upvotes in real-time
AskQuestionActivity.java	Submit new questions with subject & anonymous toggle
FacultyDashboardActivity.java	View, filter and answer questions
QuestionAdapter.java	RecyclerView adapter with upvote + answer logic
Question.java	Data model / POJO for Firestore mapping

5.DIAGRAMS


System architecture — Smart Q&A
User layer
Student
Ask · Upvote · View answers
Faculty
View · Filter · Answer
Android app layer
Login
Firebase Auth
Ask question
Subject · Anonymous
Upvote
arrayUnion / Remove
Answer
Faculty dialog
Firebase layer (backend)
Firebase Auth
Email / password login
Cloud Firestore
Questions · Real-time sync

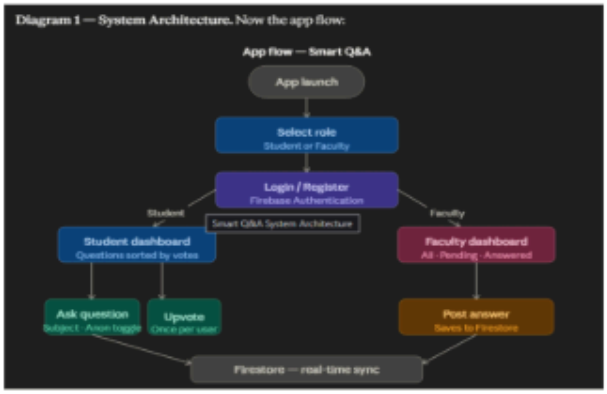


Diagram 1 — System Architecture. Now the app flow:
App flow — Smart Q&A
App launch
Select role
Student or Faculty
Login / Register
Firebase Authentication
Student
Faculty
Smart Q&A System Architecture
Student dashboard
Questions sorted by votes
Faculty dashboard
All · Pending · Answered
Ask question
Subject · Anon toggle
Upvote
Once per user
Post answer
Saves to Firestore
Firestore — real-time sync

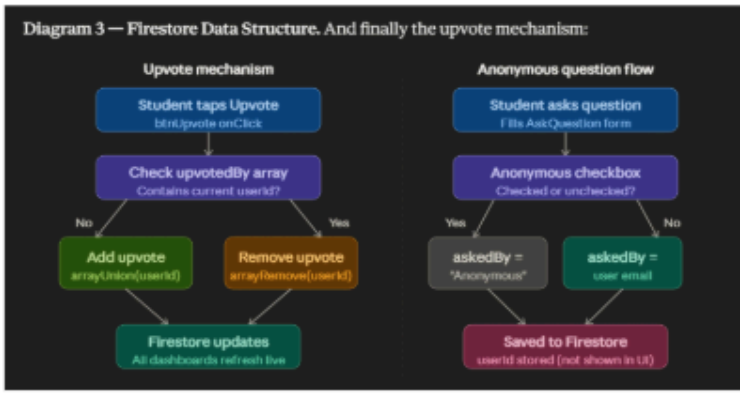


Diagram 3 — Firestore Data Structure. And finally the upvote mechanism:
Upvote mechanism
Student taps Upvote
btnUpvote onClick
Check upvotedBy array
Contains current userId?
No
Yes
Add upvote
arrayUnion(userId)
Remove upvote
arrayRemove(userId)
Firestore updates
All dashboards refresh live
Anonymous question flow
Student asks question
Fills AskQuestion form
Anonymous checkbox
Checked or unchecked?
Yes
No
askedBy =
"Anonymous"
askedBy =
user email
Saved to Firestore
userId stored (not shown in UI)

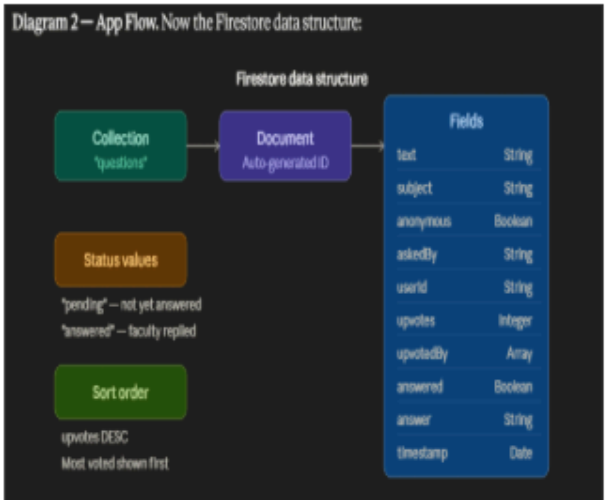


Diagram 2 — App Flow. Now the Firestore data structure:
Firestore data structure
Collection
"questions"
Document
Auto-generated ID
Fields
text String
subject String
anonymous Boolean
askedBy String
userId String
upvotes Integer
upvotedBy Array
answered Boolean
answer String
timestamp Date
Status values
"pending" — not yet answered
"answered" — faculty replied
Sort order
upvotes DESC
Most voted shown first

\

6.VIEWS & CONCLUDING REMARKS

Learning Outcomes

Building the Smart Q&A application provided hands-on exposure to several important Mobile Computing concepts. Implementing Firebase Firestore with real-time snapshot listeners was particularly insightful — it demonstrated how mobile apps can maintain live synchronization with a backend without polling, which is a fundamental advantage of modern mobile architecture.

The upvoting feature taught the importance of atomic database operations. Using FieldValue.arrayUnion() and arrayRemove() instead of reading and writing the entire array separately prevented race conditions where two students might upvote at the exact same moment. This kind of concurrency awareness is critical in multi-user mobile applications.

Challenges Faced

The most challenging aspect was managing the dual-role experience within a single codebase. Initially, having a single QuestionAdapter that could switch between student mode (upvoting enabled) and faculty mode (answer button enabled) based on a boolean flag required careful null-checking and visibility management. Another challenge was ensuring the anonymous mode genuinely hid identity — care had to be taken to confirm that the userId field, while stored in Firestore for deduplication logic, was never exposed through the UI layer.

Innovative Aspects

Unlike a simple one-directional doubt box (student writes, faculty reads), the Smart Q&A application introduces a feedback loop: students can see when their question has been answered, faculty can prioritize questions that the whole class shares (via upvotes), and the anonymous option lowers the psychological barrier to participation. These three features together create a more democratic and engaging classroom Q&A experience.

Future Enhancements

- Push Notifications: Use Firebase Cloud Messaging (FCM) to alert students when their question is answered.
- Subject-wise Chatrooms: Extend the app to include topic-specific Q&A threads.
- AI Integration: Use an on-device ML model to detect duplicate questions and auto-merge them.
- Analytics Dashboard: Provide faculty with charts showing which topics generate the most doubts over time.
- Polls & Quizzes: Allow faculty to post quick polls to gauge student understanding in real time.

Conclusion

The Smart Q&A application successfully demonstrates the practical application of Android development skills in solving a real academic problem. The project covers key Mobile Computing topics including Android Activity lifecycle, Firebase Authentication, Firestore real-time database, RecyclerView with custom adapters, Intent-based navigation, and role-based access control. The application is functional, scalable, and directly relevant to modern digital classroom needs.

This activity deepened my understanding of how mobile technology can transform even simple physical tools — like a suggestion box — into rich, interactive, and intelligent platforms. It also reinforced the value of user-centric design, particularly in choosing to implement the anonymous question feature, which directly addresses the human psychology of classroom participation.

7. REFERENCES

1. Google LLC. *Firebase Documentation — Add Firebase to your Android Project.* Firebase for Android. Available at: https://firebase.google.com/docs/android/setup [Accessed: May 2026]

2. Google LLC. *Cloud Firestore Documentation — Get Started with Cloud Firestore on Android.* Firebase Developer Documentation. Available at: https://firebase.google.com/docs/firestore/quickstart [Accessed: May 2026]

3. Google LLC. *Firebase Authentication — Authenticate with Firebase using Password-Based Accounts on Android.* Firebase Developer Documentation. Available at: https://firebase.google.com/docs/auth/android/password-auth [Accessed: May 2026]

4. Android Developers. *RecyclerView — Create dynamic lists with RecyclerView.* Android Developer Documentation. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview [Accessed: May 2026]

5. Android Developers. *Introduction to Activities — Android Developer Fundamentals.* Android Developer Documentation. Available at: https://developer.android.com/guide/components/activities/intro-activities [Accessed: May 2026]

6. Android Developers. *Connect your app to Firebase using Android Studio.* Android Developer Documentation. Available at: https://developer.android.com/studio/write/firebase [Accessed: May 2026]

Thank
YOU